EXPERIMENT 10

AIR QUALITY INDEX DATA VISUALIZATION

Beginner-friendly code explanation & practical viva guide

Topic: Data Visualization using Matplotlib **Dataset:** City_Air_Quality.csv (Delhi)

1. Experiment overview and basic theory

Problem statement: Analyze Air Quality Index (AQI) trends in a city using Matplotlib. The experiment uses time-series plots, pollutant plots, monthly aggregation, distribution comparison and scatter plots to explore air-quality data.

What is AQI? Air Quality Index is a numerical indicator used to communicate how polluted the air is and the level of health concern associated with it. Higher AQI generally indicates poorer air quality. AQI is an index, not a physical concentration unit like µg/m³.

PM2.5: Fine particulate matter with diameter 2.5 micrometres or smaller. **PM10:** Particulate matter with diameter 10 micrometres or smaller. **CO:** Carbon monoxide, a colourless gas produced by incomplete combustion. The notebook labels CO in mg/m³. **AQI_Bucket:** A category label for grouping AQI values.

Important distinction: PM2.5 and PM10 are particulate pollutants; CO is a gas. Their units and scales differ, so the notebook plots each pollutant separately rather than using one shared y-axis.

Workflow

1. Load the CSV into a Pandas DataFrame.
2. Filter rows for Delhi.
3. Convert Date to datetime and set it as the index.
4. Inspect selected columns and count missing values.
5. Plot daily AQI and individual pollutant trends.
6. Calculate monthly mean AQI, compare distributions by AQI category, and inspect AQI–pollutant relationships.

What this notebook does not do: It does not train a machine-learning model, establish causation, or calculate a formal correlation coefficient. It performs exploratory data visualization. It counts missing values but does not apply a complete missing-data treatment strategy.

Libraries: Pandas • NumPy • Matplotlib | Focus: explain what, why, output and alternatives for each code section.

2. Imports, loading and filtering the dataset

Cell 1 — importing libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

pandas (pd): A library for tabular data. It provides DataFrame and Series objects, CSV reading, filtering, date handling, grouping and missing-value operations.

NumPy (np): A library for numerical computing and arrays. It is imported here, but the shown notebook cells do not directly use np. If asked, say it is available for numerical operations but is currently unused.

Matplotlib.pyplot (plt): A plotting interface used to create and customize charts. The alias plt is a common convention.

Cell 2 — reading and filtering

```
air = pd.read_csv('City_Air_Quality.csv')
air = air[air['City'] == 'Delhi'].copy()
air.head()
```

pd.read_csv(path) reads a comma-separated file and returns a DataFrame. The file must be in the notebook's working directory or a correct path must be provided.

air['City'] == 'Delhi' creates a Boolean condition: True for Delhi rows and False for other cities. Using air[condition] keeps only rows where the condition is True.

.copy() creates a separate DataFrame copy. It helps avoid accidental chained-assignment behaviour when modifying the filtered data later.

.head() displays the first five rows by default. Use air.head(10) to show ten rows. It is a quick preview, not a complete data inspection.

What is a DataFrame?

A DataFrame is a two-dimensional labelled table with rows and columns, similar to a spreadsheet or database table. A Series is a one-dimensional labelled array, such as air['AQI'].

Cross-questions

Q. What does read_csv return?

Answer: A Pandas DataFrame.

Q. Why filter Delhi only?

Answer: The experiment is focused on one city, so filtering avoids mixing observations from different locations.

Q. What if the city value is 'delhi' in lowercase?

Answer: The exact comparison will not match it. We would normalize text, for example with air['City'].str.strip().str.lower() == 'delhi'.

Q. Why use .copy()?

Answer: To work on an independent filtered DataFrame and reduce SettingWithCopyWarning risks.

Q. Does head() modify the data?

Answer: No. It only returns a preview of the first rows.

3. Date handling, index and data inspection

Cell 3 — convert Date and set index

```
air['Date'] = pd.to_datetime(air['Date'])
air = air.set_index('Date')
```

pd.to_datetime() converts date-like strings into Pandas datetime values. Datetime values can be sorted, filtered by date and used for time-based operations such as resampling.

set_index('Date') makes Date the row index instead of an ordinary column. A DatetimeIndex is especially useful for time-series plots and resampling.

What is a time series?

A time series is data collected or recorded over time, such as daily AQI readings, hourly temperature, or monthly sales. The order of observations matters. A line plot can show changes and patterns over time.

Cell 4 — choose relevant columns and inspect

```
relevant = ['PM2.5', 'PM10', 'CO', 'AQI', 'AQI_Bucket']
air[relevant].info()
```

The list relevant stores column names. air[relevant] selects only those columns. **.info()** prints a concise summary: column names, non-null counts, data types and memory usage. It helps identify missing values and incorrect types.

Cell 5 — count missing values and preview

```
air[relevant].isna().sum()
air.head()
```

.isna() creates a Boolean table: True where a value is missing (NaN/None), False otherwise. **.sum()** counts True values column-wise because True is treated as 1 and False as 0.

Notebook detail: In a Jupyter cell containing two bare expressions, the displayed output is normally the value of the final expression only. Therefore, air.head() may be the visible output, while the missing-value count may not appear. To show both, write print(air[relevant].isna().sum()) before air.head(), or assign the count to a variable.

Cross-questions

Q. What is NaN?

Answer: A common representation of a missing or undefined numerical value.

Q. Difference between isna() and dropna()?

Answer: isna() identifies missing values; dropna() removes rows or columns containing missing values according to its settings.

Q. Does set_index delete the Date information?

Answer: No. It moves Date from a regular column into the index.

Q. Why use datetime instead of leaving dates as strings?

Answer: Datetime enables chronological operations, date-based indexing and resampling.

Q. What does non-null count mean in info()?

Answer: The number of values that are present, not missing, in that column.

4. Overall AQI trend line plot

Cell 6 — plotting daily AQI over time

```
air['AQI'].plot(figsize=(10, 4), color='purple', label='AQI')
plt.title('Delhi: AQI over time')
plt.xlabel('Date')
plt.ylabel('AQI')
plt.legend()
plt.tight_layout()
plt.show()
```

Line-by-line explanation

• **air['AQI']** selects the AQI Series from the DataFrame.

• **.plot()** draws a line plot by default for a Series. Since Date is the index, dates are used on the x-axis and AQI on the y-axis.

• **figsize=(10, 4)** sets figure width and height in inches.

• **color='purple'** sets the line colour.

• **label='AQI'** provides the name shown in the legend.

• **plt.title()** adds the chart title; **xlabel()** and **ylabel()** label the axes.

• **plt.legend()** displays labels for plotted data.

• **plt.tight_layout()** adjusts spacing so labels and titles are less likely to overlap or be clipped.

• **plt.show()** displays the figure.

How to interpret it

Look for long-term upward or downward movement, repeated peaks, sudden spikes, and periods of relatively stable values. A peak indicates a high AQI observation on that date; it does not by itself explain the cause.

Common cross-questions

Q. Why use a line chart?

Answer: Because dates are ordered and a line helps show continuous change and trends over time.

Q. What is the x-axis and y-axis?

Answer: The x-axis represents Date; the y-axis represents AQI.

Q. What does a sudden spike mean?

Answer: AQI was unusually high at that observation. Further investigation is needed to identify why.

Q. What if we use a bar chart instead?

Answer: A bar chart can compare discrete dates, but a line plot is usually clearer for a continuous time sequence.

Q. What does figsize=(10,4) mean?

Answer: A figure 10 inches wide and 4 inches tall.

Q. Can a line plot prove that pollution caused a health problem?

Answer: No. It visualizes values and patterns; it does not establish causation or health outcomes.

Remember: correlation, trend and cause are different ideas. A visual pattern is a clue for further analysis, not proof of a cause.

5. Plotting PM2.5, PM10 and CO separately

Cell 7 — for-loop and multiple pollutant plots

```
for pollutant, unit, color in [
    ('PM2.5', 'micrograms/m3', 'tomato'),
    ('PM10', 'micrograms/m3', 'steelblue'),
    ('CO', 'mg/m3', 'seagreen')
]:
    air[pollutant].plot(figsize=(10, 3),
                        color=color, label=pollutant)
    plt.title(f'Delhi: {pollutant} over time')
    plt.xlabel('Date')
    plt.ylabel(unit)
    plt.legend()
    plt.tight_layout()
    plt.show()
```

Understanding the loop

The list contains three tuples. Each tuple stores a column name, its displayed unit and a colour. The for-loop takes one tuple at a time and unpacks it into pollutant, unit and color. The indented statements execute once for each pollutant — so three separate charts are produced.

f-string: f'Delhi: {pollutant} over time' inserts the current variable value into the text. For example, when pollutant is PM2.5, the title becomes “Delhi: PM2.5 over time”.

Why separate charts?

PM2.5 and PM10 are typically expressed in micrograms per cubic metre, while this notebook labels CO in milligrams per cubic metre. Their scales and units differ. Separate charts make each pollutant's trend easier to read and avoid misleading comparisons on one y-axis.

Cross-questions

Q. What is a loop?

Answer: A programming structure that repeats a block of code for each item in a sequence.

Q. What is tuple unpacking?

Answer: Assigning tuple elements to separate variables, as in pollutant, unit, color.

Q. How many charts does this loop create?

Answer: Three, one for each tuple in the list.

Q. What is an f-string?

Answer: A formatted string that allows variable values or expressions inside curly braces.

Q. Can we add another pollutant?

Answer: Yes. Add another tuple with its column name, unit and colour, provided that column exists in the DataFrame.

Q. Why not plot every pollutant on the same axis?

Answer: Different units and scales can make one series dominate or make comparisons misleading.

Q. What if the column name is misspelled?

Answer: Pandas raises a KeyError because that column cannot be found.

Syntax reminder: the colon starts the loop block; indentation identifies the statements repeated in every iteration.

6. Monthly AQI aggregation using resample()

Cell 8 — monthly average AQI bar chart

```
monthly_aqi = air['AQI'].resample('ME').mean()
monthly_aqi.index = monthly_aqi.index.strftime('%Y-%m')
monthly_aqi.plot(kind='bar', figsize=(13, 4), color='blue')
plt.show()
```

What happens here?

resample('ME') groups a time-indexed Series into month-end frequency bins. Because Date is the DatetimeIndex, Pandas knows how to group observations by calendar month. **.mean()** calculates the average AQI within each month.

strftime('%Y-%m') converts each date-like index value into a readable year-month label, such as 2026-03. Here, it is used to make bar-chart labels compact.

kind='bar' requests a bar chart. The bar height represents the monthly mean AQI. figsize=(13,4) makes the chart wider to fit multiple month labels.

Why aggregate?

Daily data can be noisy and contain many observations. Monthly averaging summarizes each month and makes broad month-to-month comparisons easier. However, averaging can hide short pollution episodes or extreme daily values.

Version note

The notebook uses 'ME' for month-end resampling, supported by modern Pandas versions. In older Pandas versions, 'M' was commonly used. If 'ME' produces an invalid-frequency error in an older environment, check the Pandas version and use the supported month-end alias for that version.

Cross-questions

Q. What is resampling?

Answer: Changing the frequency of time-series data, usually by grouping observations into a new time interval.

Q. What does ME mean?

Answer: Month End; each group is associated with the end of a calendar month.

Q. Why mean() after resample()?

Answer: To calculate the average AQI for each month.

Q. Does monthly mean mean the AQI was constant throughout the month?

Answer: No. It is a summary average; daily values can vary considerably.

Q. Mean versus median?

Answer: Mean is the arithmetic average and is sensitive to extreme values. Median is the middle value after sorting and is more resistant to outliers.

Q. What if we use 'D' or 'YE'?

Answer: 'D' groups by day; 'YE' groups by year-end in modern Pandas frequency conventions.

7. Box plot and scatter plot

Cell 9 — compare AQI distributions by AQI_Bucket

```
air.dropna(subset=['AQI', 'AQI_Bucket']).boxplot(
    column='AQI', by='AQI_Bucket',
    rot=30, figsize=(13, 4)
)
plt.ylabel('AQI')
plt.show()
```

dropna(subset=[...]) removes rows that have missing values in either AQI or AQI_Bucket for this plot. It does not modify the original air DataFrame unless assigned back.

boxplot(column='AQI', by='AQI_Bucket') draws a separate AQI distribution for each category. **rot=30** rotates category labels by 30 degrees. A box plot shows median, quartiles, spread and possible outliers.

Box plot vocabulary

• **Median:** Middle value; the line inside the box.

• **Q1 and Q3:** 25th and 75th percentiles; the box spans the interquartile range (IQR = Q3 – Q1).

• **Whiskers:** Extend to values within the plot's whisker rule (commonly up to 1.5 × IQR from the quartiles).

• **Outliers:** Points beyond the whiskers under that rule; they are unusual relative to the distribution, not automatically errors.

Cell 10 — scatter plots of AQI against pollutants

```
for pollutant in ['PM2.5', 'PM10', 'CO']:
    air.plot.scatter(
        x=pollutant, y='AQI',
        alpha=0.3, color='orange'
    )
```

plot.scatter(x=..., y=...) draws one point per row, with pollutant concentration on the x-axis and AQI on the y-axis. **alpha=0.3** makes points partly transparent, helping reveal dense overlapping regions. The loop produces three scatter plots.

How to interpret scatter plots

An upward pattern suggests a positive association: higher pollutant values tend to occur with higher AQI. A downward pattern suggests a negative association. A random cloud suggests weak or no obvious linear association. The plot alone does not calculate a correlation coefficient and cannot prove causation.

Cross-questions

Q. What does a box plot show that a line plot does not?

Answer: It summarizes a distribution using median, quartiles, spread and potential outliers.

Q. What is IQR?

Answer: Interquartile range, calculated as Q3 minus Q1.

Q. What does alpha control?

Answer: Transparency; 0 is fully transparent and 1 is fully opaque.

Q. Why use scatter plot?

Answer: To visually inspect the relationship between two numerical variables.

Q. Does positive correlation mean one variable causes the other?

Answer: No. Correlation is association, not proof of causation.

8. High-value viva cross-questions and code improvements

General Python / Pandas / Matplotlib questions

Q. What is the difference between a library and a module?

Answer: A library is a collection of reusable code; a module is a Python file or unit that contains definitions. Matplotlib is a library and pyplot is a module/interface within it.

Q. What is the difference between a Series and DataFrame?

Answer: A Series is one-dimensional; a DataFrame is two-dimensional and has labelled rows and columns.

Q. What is Boolean indexing?

Answer: Filtering data using a True/False condition, such as air[air['City'] == 'Delhi'].

Q. What is the difference between plot() and scatter()?

Answer: plot() commonly draws connected lines for sequential data; scatter() draws independent points to show relationships between numerical variables.

Q. What does plt.show() do?

Answer: Displays the current Matplotlib figure.

Q. What is an outlier?

Answer: An observation unusually far from the rest of the data. It should be investigated, not removed automatically.

Q. What is data cleaning?

Answer: Finding and addressing issues such as missing values, duplicates, invalid types and inconsistent values.

Q. What is exploratory data analysis (EDA)?

Answer: The process of inspecting, summarizing and visualizing data to understand its structure, quality and patterns.

Limitations and improvements in this exact notebook

• **Missing values:** Cell 5 counts missing values, but the notebook does not apply a general imputation strategy. Some plotting operations may skip missing values; this should be checked deliberately.

• **Chronological order:** After converting Date, consider air = air.sort_index() before time-series analysis if the source rows are not already ordered.

• **Duplicate dates:** Check whether multiple rows exist for the same date. Do not remove them without understanding what each row represents.

• **Chart clarity:** Add titles and axis labels to the monthly bar chart, box plot and scatter plots for consistent presentation.

• **Correlation:** If asked for a numerical correlation, calculate it separately, for example air[['AQI','PM2.5','PM10','CO']].corr(). This is not currently done in the notebook.

• **Unused import:** NumPy is imported but not used in the visible code. It can be removed if not needed.

Useful additional code (only if asked to improve the experiment)

```
# Sort observations by date
air = air.sort_index()

# Show missing-value counts clearly
print(air[relevant].isna().sum())

# Numerical correlation matrix
print(air[['AQI', 'PM2.5', 'PM10', 'CO']].corr())

# Add labels to a scatter plot
air.plot.scatter(x='PM2.5', y='AQI', alpha=0.3)
plt.title('AQI vs PM2.5 in Delhi')
plt.xlabel('PM2.5 concentration')
plt.ylabel('AQI')
```

```
plt.tight_layout()
plt.show()
```

A simple 30-second explanation to say in viva

“In this experiment, I used Pandas to load the city air-quality CSV file and filtered the records for Delhi. I converted the Date column to datetime and set it as the index so that time-series operations could be performed. I then inspected the selected pollutant and AQI columns, counted missing values, and used Matplotlib to visualize daily AQI and individual pollutant trends. I also calculated monthly average AQI using resampling, compared AQI distributions across AQI categories using a box plot, and used scatter plots to visually study the relationship between AQI and PM2.5, PM10 and CO.”

Final viva tip: For every code line, remember four things: what it does, why it is used, what output it produces, and one alternative or parameter change.